IIT Madras

ONLINE DEGREE

Machine Learning Techniques
Professor Arun Rajkumar
Department of Computer Science and Engineering
Indian Institute of Technology Madras
Representation Learning: Part 1

Hello, everyone, welcome back. Today, we are going to start the proper mathematical foundations of this course, Machine Learning Techniques. And as I said, we are going to cover different aspects of machine learning techniques in this course, which include unsupervised learning, supervised learning. And, specifically, we will look at algorithms for both of these. So, what we will start with first is unsupervised learning. So, the first part of this course will cover different types of unsupervised learning.

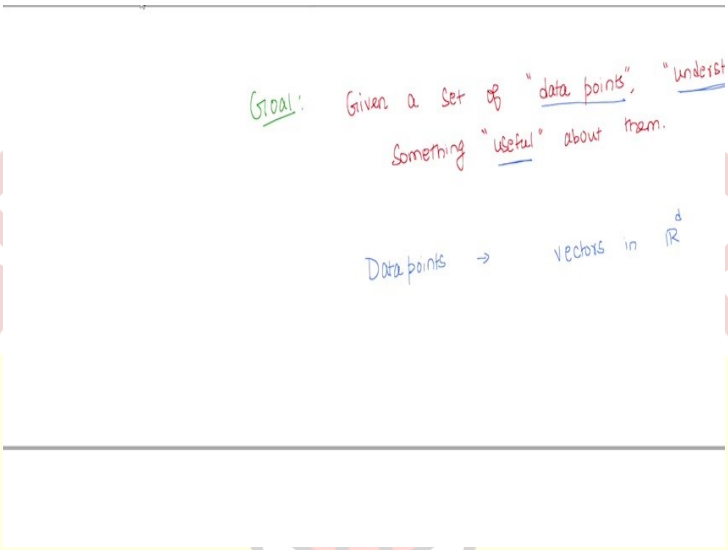
(Refer Slide Time: 0:48)

So that is what we will get started with - unsupervised learning. In particular, we will start with one subcategory of unsupervised learning, which is called as representation learning. So, what is the goal of unsupervised learning? So, what is unsupervised learning? Well, we are given a set of data points. So, we are given a set of data points, our goal is to understand something useful about the set of data points.

So, this is a very-very vague, broad goal that we want to start with. So, of course, it is vague and broad because of three different things, one is, we have to define what do you mean by given a set of data points. So, what are data points? More importantly, we need to understand, well, what does understand mean? And what does usefulness mean?

So, remember, in unsupervised learning, we are just given a bunch of data points, we are not given any supervision, we saw this before. But now if you are given just a bunch of data points, what can you understand what does it mean to say we are understanding something and more importantly, something useful about them?

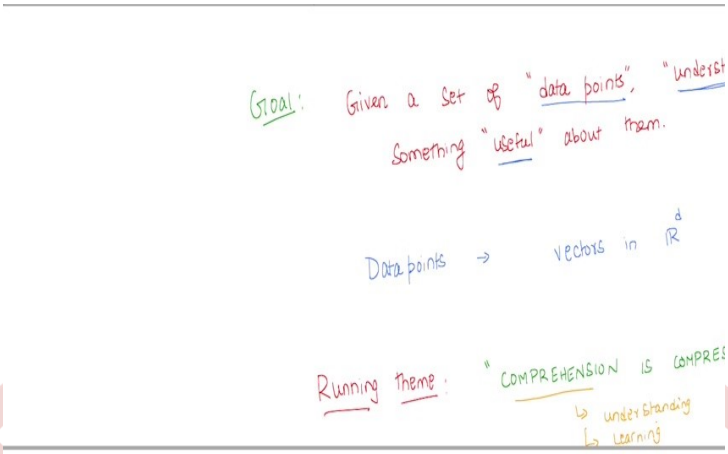
(Refer Slide Time: 2:27)



So, the easier question to answer here is the question of what are data points. Well, data points, as you may already have guessed, we are going to think of these as vectors in some D dimension. So typically, real numbers, D real numbers, these are features. For example, I might collect the height, weight, and age from 100 different people, so each person becomes a three-dimensional point, where each of the coordinates correspond to height, weight and age, in that particular order.

So, data points are clear. So, we are going to think about data points is just a bunch of vectors in the dimension. But the more important question is, what does it mean to say we understand something about this data point and what does it mean to say, we understand something useful? So, those once we fix that, then we will be able to do something useful. So, towards this, of course, there are multiple ways to approach this, we are going to start with one running theme, which I will put down and then explain what it is.

(Refer Slide Time: 3:35)



So, this running theme of what does it mean to understand will come in very handy not just for the representation learning part that we are looking at today, but in general throughout this course. So, what is the running theme that we are going to look at? Well, this is based on a very nice quote by a famous computer scientist and a philosopher George Chaitin. And he says, comprehension and it is worth writing it down, "Comprehension is compression" that is a very interesting statement and this is by George Chaitin.

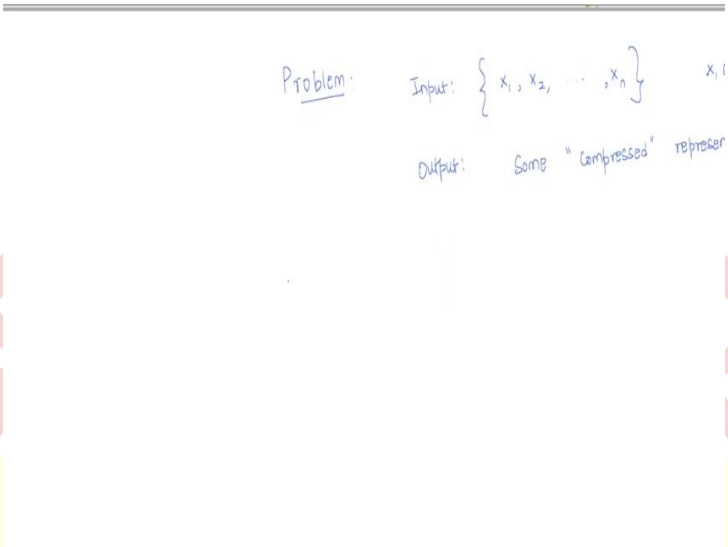
So, we want to comprehend. So, what does comprehend mean? In our context, we can think of comprehension as well understanding, so or even learning. So, you can comprehend, which means you can understand something you can learn something all these are synonymous in our context. But what does it mean to say you can comprehend something, it means that you are able to compress.

So, for instance, if you are able to compress information such that you retain only the important part of the information that can be explained to somebody else, then it means that you have necessarily understood or learned from the data. So, this is a very high-level running theme that we are going to use not just for the algorithm that we will see for unsupervised learning now, but then in some sense throughout this course.

So I may not repeat this every time, but whenever we see a new algorithm, I think there is merit in asking the question, well, what does it mean to say, we are learning from in this particular algorithm? In other words, where is the compression really happening? Ask yourself that question every time you encounter an algorithm in this course and I think that

will give you some new insights as to how these algorithms are designed and why these algorithms act in a particular way and so on.

(Refer Slide Time: 5:47)

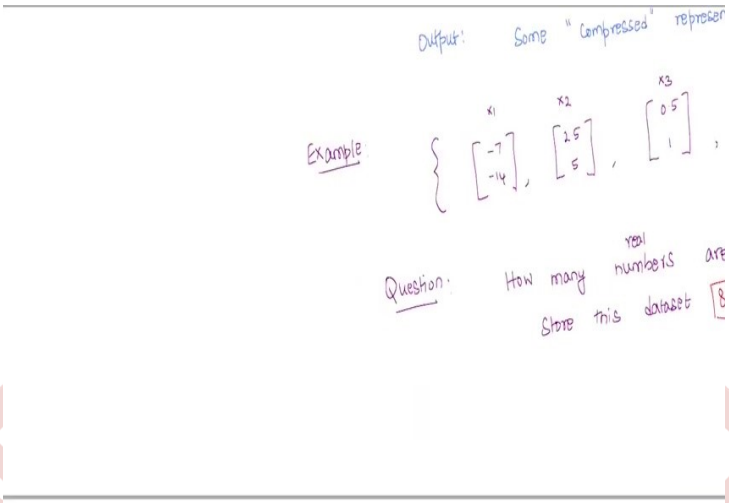


For now, we are going to start with the running theme for the problem of unsupervised learning. So, let us make that problem a little bit more precise. Let me not use yellow, let me use blue. So, here is the problem that we want to think about now. So, you have an input. Now we have decided the input is a bunch of data points and these data points can be thought of as a bunch of vectors in R^d.

Let us say we have n such vectors, each x_i is in R^d which means there are n people. From each person, let us say we collect d different numbers or these are in general data points, maybe these are n images, whatever it could be, but then we just have n different data points, which we are abstracting it out as a bunch of vectors in R^d. So, this d can be thought of as features, the number of features.

So, what is the output that we want? Well, from this data point, because we are going to think of comprehension as compression, learning as compression, we want some compressed representation of the data set. Now, what does it mean to say we can output a compressed representation of the data set, so that is the next question we have to ask ourselves. And for this, let us start with a simple example.

(Refer Slide Time: 7:08)



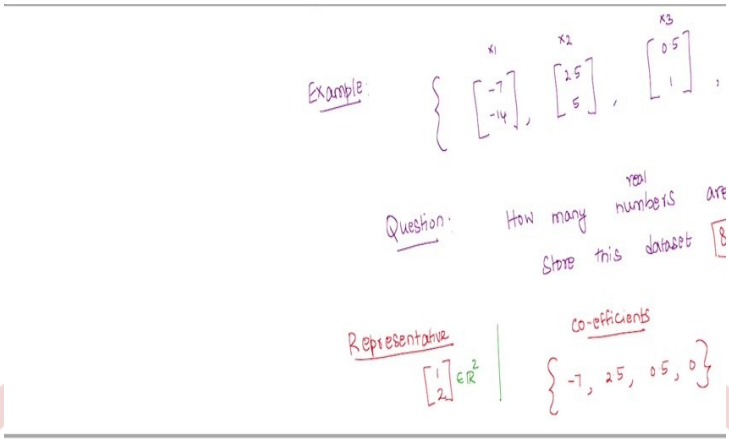
So, here is an example. Let us say I give you a data set, which looks like the following. So, maybe you have [-7 -14], this is x_1, x_2, let us say is [2.5, 5], x_3 is let us say [0.5, 1], x_4 is let us say [0, 0]. Let us say you have these four data points, I am going to ask you a question and then maybe you can pause and think about this question a bit and we will talk about what is the answer to this question in the way that we are thinking about.

So, how many numbers are needed, when you say numbers, real numbers. So, how many real numbers are needed to store this data set, let us say on a computer? So, there are 4 data points, each data point has 2 coordinates. So, how many numbers do you think are needed to store this data set? Pause and think about this, I will tell you the answer.

The naive answer to this question is that well, 4 data points to features per number, so 4 x 2, 8, so, you would need 8 numbers, you can store 8 numbers on a computer, and then well good, so that you can retrieve the data set exactly as you are seeing it here, which is good. But then is this the best that we can do, do we really have to store 8 numbers in this case or can we do better?

So, if you think about it, now look at the data set and see if there are some relationships between the features that can be exploited, that will allow us to store lesser number of values, and still be able to reconstruct this dataset exactly, pause and think about it. And now, let me tell you the answer to this, it is not too hard to see that there is a relationship between the first coordinate and the second coordinate. In particular, the first coordinate is always half the second coordinate. So in this particular data set for all the four data points.

(Refer Slide Time: 9:26)



So, which means one way you could store this data set on a computer is going to be as follows. We will store a representative point, this is one type of representing this dataset, not the only type, here is one way. So, the representative in this case, let us say is [1 2]. So, here is a single representative for the entire data set, which is [1 2], which kind of tells us that if the first feature is 1, the second feature is twice the first feature.

That is what this representative says. And now for each data point we will store what are called as coefficients, and the score sufficient for the data point 1 is going to be -7, for data point 2 is going to be 2.5, for x_3 is going to be 0.5 and x_4 is going to be 0. Now, suddenly you see that well, if you exploit the fact that the first two coordinates are related.

And if you want a representation where you have a representative, which is one vector in R^2, for the entire data set, and coefficients, which is one coefficient per representative, then suddenly we just need only 6 numbers to be stored. Now, one might ask, well is this a big savings, how much of a savings can we really achieve? So, we just went from 8 to 6, but that does not seem like a big savings.

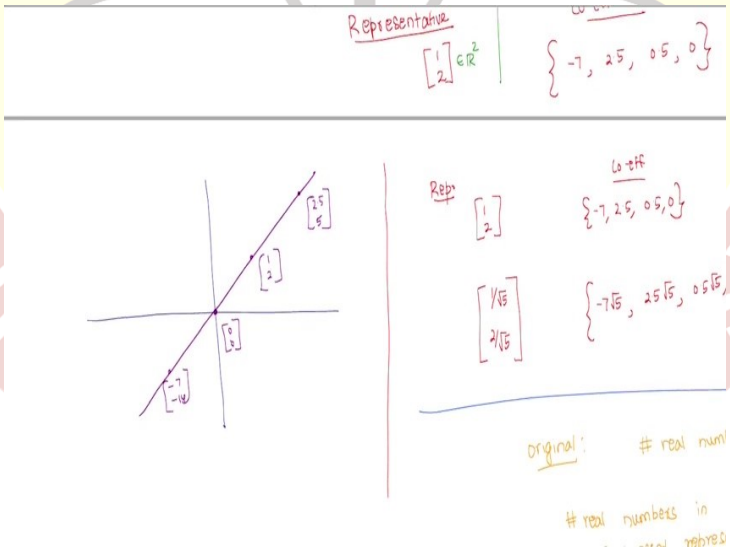
Now, if you think about it, it is going from 8 to 6 only because it is just 4 data points, but if you had a billion data points, instead of storing 2 billion numbers, now how many numbers would you have to store if you had this way of representing things? Again, you would just have one representative, which needs two numbers, one vector in R^2, and then for every data point, you just need to store one number instead of two.

So, you would have 1 billion + 2 numbers to store, + 2, because for the representative, and 1 billion, because one number per data point, as opposed to 2 billion numbers where there are two numbers per data point. So that is like a 50% almost 50% compression rate. So, which means we are able to compress this dataset by exploiting this relationship between the first and the second coordinate.

Now, we will talk more about this but then at this point, I want you to make a note of this point that using this representative, one thing that we can achieve is while using representative and the coefficients, one thing that we are able to achieve is we can reconstruct the data set exactly. Well, that might not seem like a big deal at this point, of course, that is what we can do. So, how do you reconstruct the data set?

So, if I asked you what is x_2, but then if I give you only the representatives and coefficients, how would you reconstruct x_2? You would simply take the representative and multiply it by the coefficient, in this case, 2.5 times to vector [1 2], which would give me [2.5 5], which is x_2. So, and you can do this for each of the data points, and you will get back the exact data set, so there is an exact reconstruction that is possible, using this way of looking at things.

(Refer Slide Time: 12:54)



So, now one can also think of this in a geometric fashion, I look at the same data set, but then let us say I try to plot these points, these 4 points on the plane, where would these 4 points lie, again, you should be already able to see it, if not, just pause and think about it. While I plot where these points lie. So, these points are going to lie along the line along the line where

these points would be [0 0], [-7 -14], [2.5 5], and I think maybe this is [1 2], and you have [2.5 5].

Now, all of these lies along this line, and that is not too hard to see, because the relationship is that the y coordinate is two times the x coordinate. So, y equals $2x$, is this line. So, all the points in this line. Now, even if you had other points in this line, well, you could still you could have used a representative and then reconstructed it the way we did. Now, one point to note here, again, a simple point, but then it will come out very useful as we go along.

Instead, well, remember what we did was the representative that we choose was [1 2], and the coefficients were -7, 2.5, 0.5 and 0. Now, there is nothing sacrosanct about this representative [1 2], I could have chosen a different representative and achieved the same exact reconstruction. For instance, I could have chosen any point along this line as a representative, of course, except the [0 0], point.

Any other point along this line could have been chosen as a representative. For instance, I could have chosen $[1/\sqrt{5} \quad 2/\sqrt{5}]$. For whatever reason I want to let us say choose this as my representative, it is still a valid represented because there are coefficients that I can use, which are $-7\sqrt{5}$, $2.5\sqrt{5}$, $0.5\sqrt{5}$, 0, and I still will be able to access exactly the constructor dataset.

So, the exact choice of representative is immaterial as long as the representative lies along this line. So let me make a note of that point and then again, it might seem a simple silly point at this point, but we will see the use of this as we go along. So, any vector along the line, purple line that I have drawn here, can be chosen as a representative, of course, except [0 0], which we will not allow, because if it is [0 0], then we will not be able to reconstruct anything.

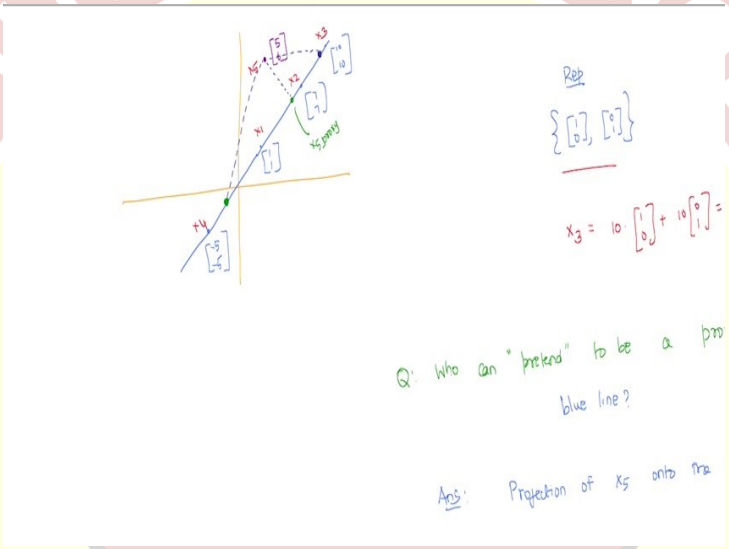
So, originally, if we did not have this representative coefficient view of things, then the number of real numbers that you would have needed to store let us say, n data points in two dimension would have been 2 x n. So, in d dimension, it would have been d xn. Now, the number of real numbers in the compressed representation is much smaller. And how many are there? Well, you should have been, you should be able to guess it now.

So, there are 2 for a single representative plus 1 for each data point. So, that is 2 + n. In general, this will be d x n. And then this would be $d + n$. So, this d is for the coefficient, if

you were thinking about in d dimension, if every other coordinate is some multiple of the first coordinate, let us say, then you just need d numbers for the representative and one number for each of the data points, so it would be interesting.

So, now that that is a huge compression that you might potentially achieve. But now, if you have been thinking about this, an obvious question that comes that should come to your mind is, well, it is fine that we are able to compress this if the data points all lied along this line. But now let us say that was not the case.

(Refer Slide Time: 17:37)



Let us say we had a situation like this. So, which is not a very nice situation for our data set here. Let us say we had this case, let us make it even easier, we have the line $y=x$ where we have a bunch of data points [1 1], I do not know, [7 7], [10 10], maybe, now this can still be compressed, 50% compression is still possible for these four data points from 8, you can go to 2 + 4, 6, all that is good.

But let us say our data set was not like this, maybe our data set had this extra point, which is [5 6], and now, the question is, can we do the compression in the way that we have done already? Specifically, the question that I am asking is, well, now if you can you represent all these data points as a single representative, and one coefficient per data point?

Well, of course you cannot, so, because all these data points do not fall on the same line, there is one data point, which is kind of, in some sense, making things harder for us. But then

if you want to exactly reconstruct this data set, well, then you have to give up the notion that can be a single representative one coefficient per data point.

Now, what else can we do? Well, one thing you can do is, instead of a single representative, you can use two representatives, and then you can think of each data point as a linear combination of these two representatives. For example, I can think of the representatives now as not just a single representative, but a set of representatives. Let us say, {[1 0], [0 1]}, are my representatives, imagine. And my coefficients would be what?

Would be to reconstruct the point [-5 -5], the coefficients would be -5, -5. To reconstruct [1 1], the coefficients will be 1, 1. To reconstruct [7 7], it would be 7, 7, [5 6] would be 5, 6, and [10 10] would be 10, 10. So, for example, how would I reconstruct the x? Let us say this is x_1, x_2, x_3, x_4, x_5. If I had to reconstruct x_3, how would I do that? Well, I look at the coefficients that are two coefficients.

One corresponding to each of the representatives. So, this would be 10 [1 0]+10 [0 1], that gives me [10 10]. Now we can check, you can do this for every data point. But now, the question, is this a great idea? Perhaps not, because what we have done is that we have increased the number of data points that we need number of coefficients that we need to store per data point from 1 to 2.

So, which means if I had to store these numbers, I would anyway store two number per data point, in addition I will also store four numbers for the representative. So initially, if it was n numbers, we would have stored 2n numbers, but now we are going to store $2n+4$, that is not really compression at all. So, we are not really compressing we are in fact, increasing the number of data points.

Now, that means that this is also not a good idea. So, increasing the number of representatives for this data set does not seem like a great idea, either. But then if you do not increase the number of representative for this dataset, then we know that you cannot exactly reconstruct this dataset. So, you have to give up one of these, what are these? One is exact reconstruction, the other is increasing the number of representatives.

So well, if you need exact reconstruction, it looks like you need to increase the number of representatives. So, the only way you can possibly do compression here, then it looks like is that you need to give up the notion that you need exact reconstruction. So, let us say we do

that. So now, I do not really need exact reconstruction, which means that I know all these four points are all on this nice line, and this x_5 is not on this line.

If I do not have to reconstruct this exactly, which means what does that equivalently mean? It means that I need to somehow find a proxy for this $x_5$along this line along this blue line. So, if I find a proxy for this x_5, let us say at some point here is a proxy for this line, maybe this is x_5 proxy, then I can imagine as if my data set only had x_5 proxy, along with x_1, x_2,x_3,x_4 and I can forget about x_5.

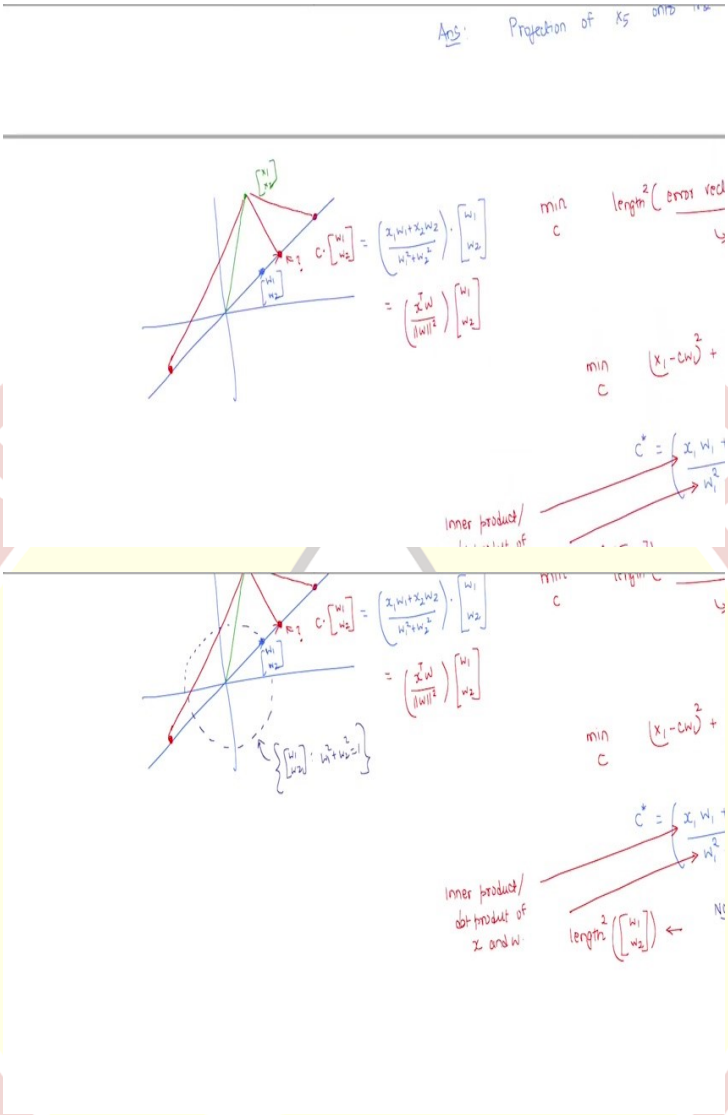
Now, because all these points lie along the line, I can use a single representative and under coefficients per data point, and compression is still possible. Now, that means we need to find a proxy for this data point along this blue line. So, how can we find this proxy? Well, if you have looked at linear algebra, before you already, perhaps know the answer to this.

Well, what could be a good proxy, a proxy the best proxy for x_5 along the blue line would be that point which for which we lose the least. So, in other words, so if I choose for instance this point as the proxy, then what do I lose? Well, I lose this bit. So, whereas if I choose this point as the proxy, I lose this bit. Now if I choose the green point as a proxy, I lose this bit.

Now, the proxy should be chosen such that I lose the least. So, which means that the length of the vector, which should be added to this proxy to get the original point should be as small as possible. Now, how do we choose that? Well, again, if you have seen linear algebra, this is just projection of x_5 on to this blue line. So, now the question that we are asking is, who can pretend to be a proxy for x_5 along the blue line?

Of course, the answer to this is projection of x_5 on to the blue line. Well, projection is just finding that point which is closest to x_5 along the blue line. So, how do we find that point? Well, of course, for people who are conversant, linear algebra might already know the answer to this. Nevertheless, just to keep it self-contained, I will go with this. So basically, this is what we want to do.

(Refer Slide Time: 24:45)



So, you have a line. Let us say we have the vector w_1, w_2, which we think of as the representative for this line, which means this line is just the set of all vectors which are scalar multiples ofw_1,w_2, $w_1$and w_2 is both not equal to 0. And now you have a point, which is, let us say x_1, x_2. Now the question we are asking is, well, what is the projection of x_1,x_2, along the line given by w_1, w_2, what is this?

Well, because of the fact that this red point is along the blue line, it has to be some constant times w_1, w_2. Now, which is that constant as you vary this constant, this point is going to move along this blue line. And for some value of this constant, we will find that this error piece is as small as possible, the length of the error is as small as possible, which means we

can set this as an optimization problem where we want to minimise with respect to c the length, or the length square of the error vector.

But what is the error vector? If the point that you want to project on this x_1, x_2, on the line that you want to project along is w_1, w_2. And if the point is c times w_1, w_2, the error vector itself is just what should be added to c, w_1, w_2, to get x_1, x_2, well, what should you add, you should add ( ) $(x - cw_1),(x - cw_2)$ And the length square of this is just $(x - cw_1)^2$ $+(x - cw_2)^2$.

Now, you want to find the c such that this value is as small as possible. So, as I move c, if I choose a different c, I might get this point where then where I am kind of measuring this line, if I choose a different c, negative c, I might get this point where the length that I am considering caring about is this length. So now, I want to choose a c such that the length is as small as possible, which means I need to minimise this function.

Now, here is an exercise, take the take the derivative of this function with respect to c, equate it to 0 and see what we get, I am not going to do that derivation, it is a very simple derivation, I will leave that as an exercise. But then if you do that, you will observe that c^* will have the following form $\frac{(xw_1+x_2w_2)}{(w_1^2+w_2^2)}$. So, remember this is a scalar.

So, it just says that what should I scale my vector w_1, w_2 by to get to a point along the line, blue line, which is closest to the point x_1, x_2. And that scalar, of course, has to depend both on w_1, w_2, and it also has to depend on the original point x_1, x_2, and a sanity check, you will see that it depends on both. So, basically what is this c?

This is c times w_1, w_2 is actually from our derivation $\frac{(xw_1+x_2w_2)}{(w_1^2+w_2^2)}$, this is a vector this is a scalar. So, you are multiplying that scalar for both the both the numerator and the denominator. So, it is worthwhile to notice the numerator and the denominator separately. So, what are these, so, how do we interpret this numerator and denominator.

Now, the numerator or let us first look at the denominator, the denominator is $(w_1^2 + w_2^2)$, but what is that we know that that is just length square of the vector w_1, w_2. So, this length is just the length of a vector is just the norm. So, w_1, w_2, is by Pythagoras theorem is $\sqrt{w_1^2 + w_2^2}$ and the length square will be $(w_1^2 + w_2^2)$.

And we also should be able to recognize what the numerator is? The numerator is just the inner product or the dot product or dot product of x and w. So, this is what we have done the derivation is for two dimensions, but then the same would go through for higher dimension as well. So, for higher dimension we can say in general that c^* is just put it here only.

So, c^* can be written as $\frac{x^T w}{\|w\|^2}[w_1 w_2]$

, $x^T w$ of course, is just a notation for $x_1 w_1 + x_2 w_2$ and $\|w\|$ is just the definition for length. This is length square of w_2, of course, this is $l2$ norm but then I am not really going to explicitly say that here. Okay, good. So, now, the c^* itself looks a bit messy, it has a numerator and a denominator.

We can perhaps simplify this a little bit more by making use of the observation that we did earlier that if you want to do this compression business, then you can pick any vector along the blue line to represent the blue line, we picked some w_1 and w_2. Now, we could we could have picked any vector along this w_1 along this blue line and our c will adjust accordingly. So, because the c depends on w_1, w_2 andx_1, x_2, of course, now, if you pick a different vector, the c^* that you will get will depend on w accordingly.

Now, because of this, what we are going to do and what is what might be easier is that we can always pick let me make this as a note can always pick w_1, w_2, such that length of $[w_1\ w_2]$ equals 1. So, we could have take this w_1, $w_2$such that this lies on the unit circle. Well, what is what does it mean to say it lies on the unit circle? This is just a set ofw_1, w_2, such that$w_1^2 + w_2^2$=1 the length is 1.

Now, what is the advantage of doing that? Well, the advantage of doing that, is that, so, which implies the c^* is just $x^T w[w_1 w_2]$. So, the denominator this 1. So, because we choose a representative to have length 1, now, c^* would not have would the denominator of c^* is 1. So, we do not have to keep track of the length of w there, because we know our understanding is that we are going to represent lines as using representatives which have length 1.

So, then c^* is just$(x^T w)w$, well, w itself is a vector, which is$[w_1\ w_2]$. Okay, good. So that is good. So basically, what we have done so far is the following. So, we said initially, that if

you had a bunch of data points, such that all of them lined up along the line, then you can choose a representative and a coefficient for each of these data point.

But now we then said that, hey, only four points lie along the line, the first point does not lie along the line, then we looked for a proxy on this line and we said that well, you can just project this point onto this line, and then you are done. Which means that we need to know where this point lies along this line. And that just is given by $x^T w$, multiplied by the vector w. That is nice, but it is still not completely does not complete our requirements.